

Servicio de Enseñanza Nacional de Aprendizaje SENA
Facultad de Ingeniería de Software

Informe técnico AA3-EV01 – Importech S.A.S – Sentencias DDL y DML

Juan Diego Castañeda González

32597794–Construcción de Bases de datos con MySQL

Docente. Maryuris Esther Oliveros Rivera

20 de septiembre de 2026

Índice general

1 Resumen 2

2 Introducción 3
2.1 Objetivos del Trabajo 3

3 Metodología 4

4 Procedimiento 5
4.1 Implementación de la base de datos, tablas y columnas con DDL . . . 5
4.1.1 Inicio de conexión 5
4.1.2 Creación y acceso a la Base de Datos 5
4.1.3 Creación de las tablas y columnas 6
4.1.4 Adición de columnas y claves foraneas 7
4.2 Inserción de datos con DML 9

5 Resultados 14

6 Conclusiones 15

7 Recomendaciones 16

8 Bibliografía 17

1. RESUMEN

El presente informe detallará el proceso del diseño y estructuración del **modelo físico relacional** para la implementación de la base de datos a la empresa comercializadora **Import Tech S.A.S**[2]. Según la necesidad de automatizar y optimizar el control de inventarios, registro de ventas y productos para clientes y proveedores, se realizará en la metodología el uso de los lenguajes **SQL (Structured Query Language)** y sus abreviados de ejecución de sentencias: **Definición de datos (DDL)** y **manipulación de datos (DDL/DML)**. Como resultado, se consolidó una base de datos normalizada y robusta que garantiza la integridad referencial a través de llaves primarias y foráneas. Concluyendo que la arquitectura implementada centraliza las operaciones comerciales de manera eficiente, reduciendo márgenes de error y sirviendo como base tecnológica escalable para la toma de decisiones estratégicas de la organización.

- **Palabras clave**: Sistema de información, Base de datos relacional, Lenguaje de definición de datos (DDL), Lenguaje de manipulación de datos (DML), Importech S.A.S.

2. INTRODUCCIÓN

En el entorno empresarial las empresas comercializadoras de productos tecnológicos requieren sistemas eficientes que garanticen la disponibilidad y el control de sus operaciones. Importech S.A.S., es una organización con trayectoria en la distribución de computadoras, partes y suministros, ha experimentado un incremento significativo en su actividad comercial, lo que evidenció la necesidad de optimizar la gestión integral de sus procesos de ventas, clientes, proveedores y existencias en stock.

2.1. Objetivos del Trabajo

- **Objetivo General**: Diseñar e implementar el modelo físico relacional para el nuevo sistema de información de Importech S.A. mediante el uso de sentencias SQL (DDL y DML), garantizando la integridad y consistencia transaccional.
- **Objetivo Específicos**:
    1. Estructurar las tablas normalizadas del sistema (categorías, proveedores, clientes, productos, ventas y detalles) aplicando restricciones de integridad referencial (`PRIMARY KEY` y `FOREIGN KEY`).
    2. Ejecutar sentencias de manipulación de datos (DML) para poblar los registros base y simular operaciones transaccionales de control de inventario y facturación.
    3. Validar el funcionamiento del modelo mediante consultas estructuradas que verifiquen el cumplimiento de las reglas del negocio de la compañía.

3. METODOLOGÍA

El desarrollo del trabajo se ejecutó bajo un enfoque técnico, estructurado y documentado en tercera persona e impersonal, siguiendo las fases metódicas orientadas a la ingeniería de software y bases de datos:

1. **Fase de Análisis de Requerimientos**: Se recopilaron las necesidades operativas de Importech S.A.S., identificando las entidades críticas del negocio: Proveedores, Clientes, Categorías, Productos, Ventas y los detalles asociados a las transacciones, así como la gestión de atributos multivaluados (teléfonos de contacto).

2. **Modelado Conceptual y Lógico**: A partir de los requerimientos, se definieron las entidades, atributos y relaciones de cardinalidad (1:N y N:M), estructurando el diagrama relacional lógico normalizado hasta la tercera forma normal (3NF) para evitar redundancias y anomalías de actualización.

3. **Diseño del Modelo Físico**: Se establecieron los tipos de datos óptimos para cada atributo (ej. `ITEM`, `VARCHAR`, `DECIMAL`, `DATE`) y se determinaron las restricciones de integridad correspondientes en el **Sistema de Gestión de Bases de Datos (SGBD).**

4. PROCEDIMIENTO

En esta sección se plasmará el procedimiento paso a paso para la creación e implementación de la base de datos mediante scripts SQL sus tablas y columnas mediante el **Lenguaje de Definición de Datos (DDL)** con la sentencia `CREATE` y para la adición de nuevas columnas y llaves foraneas se realizará con `ALTER`. También se realizará los procesos de inserción, lectura, actualización y eliminación de los valores de los campos mediante el **Lenguaje de Manipulación de Datos (DML)**.

4.1. Implementación de la base de datos, tablas y columnas con DDL

4.1.1. Inicio de conexión

Lo primero en realizar fue el ingreso al servidor de **Sistema de Gestión de Bases de Datos (SGDB)** de MySQL ya generado por la empresa en el ámbito de desarrollo y conectarse con un uno de sus usuarios ya existentes con altos privilegios al realizar las pruebas en el ámbito de desarrollo para después migrarlo a producción.

- Conexión con el servidor:

```
mysql -h [servidor-sql] -u [usuario] -p
```

- Salida de comando al ingresar correctamente al servidor:

```
Type 'help;' or '\h' for help. Type '\c' to clear
   the current input statement.
mysql>
```

4.1.2. Creación y acceso a la Base de Datos

Desde la herramienta de interfaz gráfica de Workbench conectado directamente al servidor, se procedió en ejecutar el siguiente comando para la creación y acceso a la base de datos:

- Creación y acceso a la base de datos:

```
CREATE DATABASE import_tech;
USE import_tech;
```

4.1.3. Creación de las tablas y columnas

1. Creación de las tablas padre:

```
CREATE TABLE categoria(
    id_categoria INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(45) NOT NULL,
    descripcion VARCHAR(75)
);
CREATE TABLE proveedor (
    id_proveedor INT PRIMARY KEY AUTO_INCREMENT,
    NIT VARCHAR(20) UNIQUE NOT NULL,
    nombre VARCHAR(45) NOT NULL,
    NIP VARCHAR(20) UNIQUE NOT NULL,
    web VARCHAR(125) UNIQUE,
    dir_ciudad VARCHAR(45) NOT NULL,
    dir_barrio VARCHAR(45) NOT NULL,
    dir_calle VARCHAR(45) NOT NULL,
    dir_numero VARCHAR(45) NOT NULL
);

CREATE TABLE cliente (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(45) NOT NULL,
    NIP VARCHAR(20) UNIQUE NOT NULL,
    web VARCHAR(255) UNIQUE,
    dir_ciudad VARCHAR(45) NOT NULL,
    dir_barrio VARCHAR(45) NOT NULL,
    dir_calle VARCHAR(45) NOT NULL,
    dir_numero VARCHAR(45) NOT NULL
);
```

2. Creación de las tablas hija:

```
CREATE TABLE productos (
    id_producto INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(45) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0
);

CREATE TABLE ventas (
    n_factura INT PRIMARY KEY AUTO_INCREMENT,
    descuento DECIMAL(10,2),
    monto_final DECIMAL(10,2),
    fecha DATETIME
);
```

3. Creación de la tabla intermedia:

```
CREATE TABLE detalle_venta (
cantidad_unidad INT NOT NULL DEFAULT 0,
monto_total DECIMAL(10,2) NOT NULL
);
```

4. Creación de las tablas teléfono de atributos multivaluados para cliente y proveedor:

```
CREATE TABLE telefono_proveedor (
    id_telefono INT PRIMARY KEY,
    numero_telefono VARCHAR(10)
);

CREATE TABLE telefono_cliente (
    id_telefono INT PRIMARY KEY,
    numero_telefono VARCHAR(10)
);
```

4.1.4. Adición de columnas y claves foraneas

Para esta sección se uso la instancia `ALTER` demostrando el uso alterno de crear las columnas para su respectiva llave foranea. En este caso, se ordenará según su cardinalidad.

1. **Cardinalidad: Uno a Muchos (1:N)**

   - **TABLA** `ventas` **:** Relación cliente a ventas

     ```
     ALTER TABLE ventas
     ADD COLUMN cliente_id_cliente INT NOT NULL;

     ALTER TABLE ventas
     ADD CONSTRAINT fk_venta_cliente
     FOREIGN KEY (cliente_id_cliente)
     REFERENCES cliente(id_cliente);
     ```

   - **TABLA** `productos` **:**Relación productos a categoria

     ```
     ALTER TABLE productos
     ADD COLUMN categoria_id_categoria INT NOT NULL;

     ALTER TABLE productos
     ADD CONSTRAINT fk_producto_categoria
     FOREIGN KEY (categoria_id_categoria)
     REFERENCES categoria(id_categoria);
     ```

- **TABLA** `productos` **:** Relación productos a proveedor

```
ALTER TABLE productos
ADD COLUMN proveedor_id_proveedor INT NOT NULL;

ALTER TABLE productos
ADD CONSTRAINT fk_producto_proveedor
FOREIGN KEY (proveedor_id_proveedor)
REFERENCES proveedor(id_proveedor);
```

- **TABLA** `telefono_proveedor` **:** Atributo compuesto de varios números telefónicos

```
ALTER TABLE telefono_proveedor
ADD COLUMN proveedor_id_proveedor INT NOT NULL;

ALTER TABLE telefono_proveedor
ADD CONSTRAINT fk_telefono_preveedor
FOREIGN KEY (proveedor_id_proveedor)
REFERENCES proveedor(id_proveedor);
```

- **TABLA** `telefono_cliente` **:** Atributo compuesto de varios números telefónicos

```
ALTER TABLE telefono_cliente
ADD COLUMN cliente_id_cliente INT NOT NULL;

ALTER TABLE telefono_cliente
ADD CONSTRAINT fk_telefono_cliente
FOREIGN KEY (cliente_id_cliente)
REFERENCES cliente(id_cliente);
```

2. **Cardinalidad: Varios a Muchos (N:M)**

- **TABLA INTERMEDIA** `detalle_venta` **:**

```
-- Primera llave foranea hacia ventas de n_factura

ALTER TABLE detalle_venta
ADD COLUMN ventas_n_factura INT NOT NULL;

ALTER TABLE detalle_venta
ADD CONSTRAINT fk_detalle_ventas
FOREIGN KEY (ventas_n_factura)
REFERENCES ventas(n_factura);


-- Segunda llave foranea hacia productos
```

```
ALTER TABLE detalle_venta
ADD COLUMN productos_id_producto INT NOT NULL;

ALTER TABLE detalle_venta
ADD CONSTRAINT fk_detalle_producto
FOREIGN KEY (productos_id_producto)
REFERENCES productos(id_producto);
```

3. Agregar la propiedad PK a las FK para añadir la **integridad relacional** en estas columnas, evitando la duplicidad de `IDs` y que los valores no lleguen a ser nulos:

```
ALTER TABLE detalle_venta ADD PRIMARY KEY (
   ventas_n_factura, productos_id_producto);
```

Nota: esto solo aplica a las tablas intermedias al no tener una `PK` principal.

“‘latex

4.2. Inserción de datos con DML

En esta sección se aplicará el **Lenguaje de Manipulación de Datos (DML)** para realizar las operaciones de inserción, consulta, actualización y eliminación de registros sobre las tablas de la base de datos `import_tech`. Estas operaciones permiten manipular los datos almacenados mediante las sentencias `INSERT`, `SELECT`, `UPDATE` y `DELETE`.

1. **Inserción de datos mediante** `INSERT`

   Para realizar la inserción de los registros se tuvo en cuenta la dependencia existente entre las tablas. Primero se insertaron los registros de las tablas padre y posteriormente los registros de las tablas que dependen de ellas.

   - **TABLA** `categoria`**:** Inserción de las categorías de productos.

```
INSERT INTO categoria (nombre, descripcion) VALUES
('Portátiles', 'Computadores portátiles de uso
   personal y corporativo'),
('Periféricos', 'Teclados, mouses, diademas y
   accesorios');
```

   - **TABLA** `proveedor`**:** Inserción de los proveedores.

```
INSERT INTO proveedor
(NIT, nombre, NIP, web, dir_ciudad, dir_barrio,
   dir_calle, dir_numero)
VALUES
('800111222-1', 'TechDistribuciones', '11111111',
 'www.techdist.com', 'Bogotá', 'Chapinero', 'Calle
    67', '14-22'),
('800333444-2', 'Global PC Parts', '22222222',
 'www.globalpc.com', 'Medellín', 'El Poblado', '
    Carrera 43A', '1-50');
```

- **TABLA** `cliente` **:** Inserción de los clientes.

```
INSERT INTO cliente
(nombre, NIP, web, dir_ciudad, dir_barrio, dir_calle
   , dir_numero)
VALUES
('Juan Pérez', '1098765432', 'www.juanp.com',
 'Bogotá', 'Suba', 'Calle 145', '104-20'),
('María Gómez', '1098123456', NULL,
 'Medellín', 'Belen', 'Calle 30', '80-15');
```

- **TABLA** `productos` **:** Inserción de productos asociados a categorías y proveedores existentes.

```
INSERT INTO productos
(nombre, precio, stock, categoria_id_categoria,
   proveedor_id_proveedor)
VALUES
('Portátil Lenovo ThinkPad', 3500000.00, 15, 1, 1),
('Mouse Inalámbrico Logitech', 85000.00, 100, 2, 2);
```

- **TABLA** `ventas` **:** Inserción de las ventas asociadas a clientes existentes.

```
INSERT INTO ventas
(descuento, monto_final, fecha, cliente_id_cliente)
VALUES
(0.00, 3585000.00, '2023-10-01 10:30:00', 1),
(10000.00, 340000.00, '2023-10-02 11:15:00', 2);
```

- **TABLA INTERMEDIA** `detalle_venta` **:** Inserción de los registros correspondientes a la relación N:M entre ventas y productos.

```
INSERT INTO detalle_venta
(ventas_n_factura, productos_id_producto,
   cantidad_unidad, monto_total)
VALUES
(1, 1, 1, 3500000.00),
(1, 2, 1, 85000.00);
```

- **TABLA** `telefono_cliente`**:** Inserción de los números telefónicos asociados a los clientes.

```
INSERT INTO telefono_cliente
(id_telefono, numero_telefono, cliente_id_cliente)
VALUES
(1, '3101112233', 1),
(2, '3204445566', 1);
```

- **TABLA** `telefono_proveedor`**:** Inserción de los números telefónicos asociados a los proveedores.

```
INSERT INTO telefono_proveedor
(id_telefono, numero_telefono,
   proveedor_id_proveedor)
VALUES
(1, '3001234567', 1),
(2, '3017654321', 1);
```

2. **Consulta de datos mediante** `SELECT`

   Para verificar y consultar la información almacenada en la base de datos, se ejecutaron diferentes sentencias `SELECT` utilizando operaciones de unión, ordenamiento, funciones de agregación y filtros.

   - **Consulta de datos mediante** `JOIN` para relacionar clientes con sus respectivas facturas.

```
SELECT c.nombre, v.n_factura
FROM cliente AS c
JOIN ventas AS v
    ON c.id_cliente = v.cliente_id_cliente;
```

   - **TABLA** `productos`**:** Consulta de los diez productos con mayor precio.

```
SELECT nombre, precio
FROM productos
ORDER BY precio DESC
LIMIT 10;
```

   - **TABLA** `productos`: Conteo de la cantidad total de productos registrados.

```
SELECT COUNT(*) AS total_productos
FROM productos;
```

   - **TABLA** `ventas`**:** Cálculo del ingreso total de las ventas.

```
SELECT SUM(montoFinal) AS ingreso_total
FROM Venta;
```

- **TABLA** `productos`**:** Cálculo del precio promedio de los productos.

```
SELECT AVG(precio) AS precio_promedio
FROM productos;
```

- **TABLA** `productos`**:** Obtención del precio máximo y mínimo.

```
SELECT MAX(precio) AS precio_maximo,
       MIN(precio) AS precio_minimo
FROM productos;
```

- **TABLA** `productos`**:** Filtrado de productos mediante el operador `BETWEEN`.

```
SELECT nombre, precio
FROM Producto
WHERE precio BETWEEN 50000 AND 200000;
```

- **TABLA** `cliente`**:** Búsqueda de clientes cuyo nombre comienza con la letra “A”.

```
SELECT *
FROM cliente
WHERE nombre LIKE 'A%';
```

- **TABLA** `productos`**:** Filtrado de productos mediante el operador `IN`.

```
SELECT *
FROM productos
WHERE categoria_id_categoria IN (1, 3, 5);
```

3. **Actualización de datos mediante** `UPDATE`

   La sentencia `UPDATE` permite modificar los valores almacenados en registros específicos de las tablas.

   - **TABLA** `productos`**:** Actualización del precio de un producto específico.

```
UPDATE productos
SET precio = 1250000
WHERE id_producto = 5;
```

   - **TABLA** `cliente`**:** Actualización de la dirección de un cliente específico.

```
UPDATE cliente
SET dir_calle = 'Calle 100 # 15-45',
    dirCiudad = 'Bogotá'
WHERE id_cliente = 5;
```

4. **Eliminación de datos mediante** `DELETE`

   Finalmente, se realiza la eliminación de un registro específico mediante la sentencia `DELETE`, utilizando una condición que identifica el registro que será eliminado.

   - **TABLA** `productos`**:** Eliminación de un producto específico del inventario.

```
DELETE FROM productos
WHERE id_producto = 12;
```

5. RESULTADOS

La implementación del modelo físico relacional para Importech S.A.S. se ejecutó de manera exitosa en el entorno de desarrollo utilizando el motor de base de datos MySQL[5]. A partir de la ejecución de los scripts mediante el Lenguaje de Definición de Datos (DDL) y el Lenguaje de Manipulación de Datos (DML), se obtuvieron los siguientes resultados:

- **Estructuración de la base de datos**: Se crearon satisfactoriamente las tablas normalizadas principales y dependientes correspondientes a categorías, proveedores, clientes, productos, ventas, detalle de venta y tablas de atributos multivaluados para teléfonos. Para cada atributo se asignaron de manera óptima los tipos de datos requeridos, tales como `VARCHAR` y `DECIMAL`.
- **Consolidación de la integridad referencial**: Mediante la instrucción `ALTER`, se establecieron las relaciones lógicas de cardinalidad (1:N y N:M) a través de llaves foráneas[1], interconectando las entidades de ventas con clientes, los productos con sus respectivas categorías y proveedores, y las tablas intermedias.
- **Validación transaccional**: La inserción de registros base permitió simular un entorno operativo real de facturación y control de inventario[2]. Las pruebas realizadas mediante sentencias `SELECT` (como el cálculo del ingreso total o el precio promedio), junto con las operaciones de actualización (`UPDATE`) y eliminación (`DELETE`), verificaron el correcto cumplimiento de las reglas del negocio.

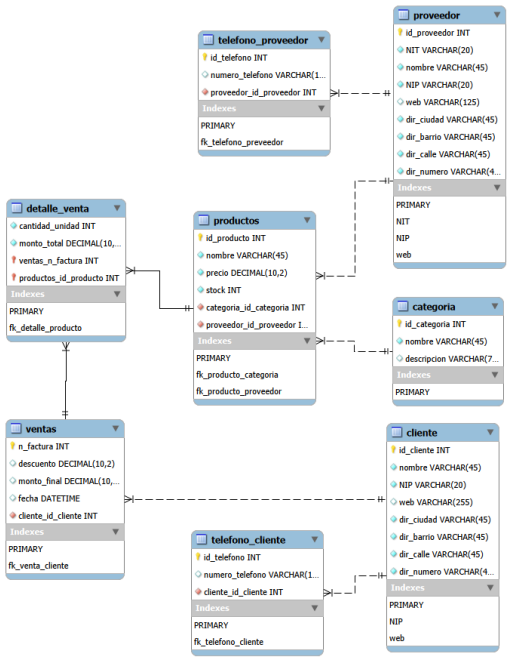


Figura 1. Modelo Relacional definitivo con Ing. Inversa de EER de Importech S.A.S.

6. CONCLUSIONES

- Se logró diseñar e implementar íntegramente el modelo físico relacional para el sistema de información de Importech S.A., garantizando un control optimizado y eficiente sobre sus operaciones comerciales[2].
- La aplicación metódica del modelado conceptual y lógico, estructurado hasta la tercera forma normal (3NF), evitó anomalías de actualización y redundancias en los datos[1].
- El despliegue a través del lenguaje DDL permitió definir una arquitectura robusta, la cual asegura la consistencia de los datos mediante el uso estricto de llaves primarias (`PRIMARY KEY`) y foráneas (`FOREIGN KEY`)[5].
- Las transacciones ejecutadas mediante el lenguaje DML demostraron que la base de datos centraliza los flujos de información de manera estable. Esto confirma que el sistema resultante sirve como una base tecnológica escalable, ideal para facilitar la toma de decisiones estratégicas dentro de la organización.

7. RECOMENDACIONES

Con base en los resultados obtenidos y la estructura implementada, se sugieren las siguientes acciones para el entorno de producción del sistema[3]:

- **Implementación de respaldos automatizados:** Se sugiere establecer políticas y rutinas periódicas de copias de seguridad (*backups*) en el servidor MySQL para salvaguardar la información transaccional ante eventuales fallos de hardware o software[4].
- **Optimización de consultas (Indexación):** A medida que el volumen de datos comerciales crezca, se recomienda la creación de índices adicionales en columnas de búsqueda frecuente (como fechas de ventas o nombres de productos) para mejorar los tiempos de respuesta[3].
- **Desarrollo de capa de aplicación (Interfaz):** Se aconseja construir una interfaz gráfica de usuario (GUI) o API que permita a los cajeros y administradores interactuar de manera segura con el motor de base de datos, restringiendo la manipulación directa del código SQL[6].

8. BIBLIOGRAFÍA

[1] Edgar F. CODD. «A Relational Model of Data for Large Shared Data Banks». En: *Communications of the ACM* 13.6 (1970), págs. 377-387.

[2] ECOSISTEMA DE RECURSOS EDUCATIVOS DIGITALES SENA. *Caso de estudio: Import Tech*. Mayo de 2021. URL: https://www.youtube.com/watch?v=Ih7CYH73mrM (visitado 19-09-2026).

[3] Ramez ELMASRI y Shamkant B. NAVATHE. *Fundamentals of Database Systems*. 7th. Pearson, 2015.

[4] Baron SCHWARTZ, Peter ZAITSEV y Vadim TKACHENKO. *High Performance MySQL: Optimization, Backups, and Replication*. 3rd. O'Reilly Media, 2012.

[5] Abraham SILBERSCHATZ, Henry F. KORTH y S. SUDARSHAN. *Database System Concepts*. 7th. McGraw-Hill Education, 2020.

[6] Ian SOMMERVILLE. *Software Engineering*. 10th. Pearson, 2015.